



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 20: Patrones Creacionales (Motivación)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

16 de Octubre, 2025

Cuestionario cap 4 y 5

- Existen comentarios buenos.

- Existen comentarios buenos.
- Si tu código está limpio entonces no debería tener ningún comentario.

- Existen comentarios buenos.
- Si tu código está limpio entonces no debería tener ningún comentario.
- Es una buena práctica comentar todos los métodos de nuestras clases con una descripción de lo que hacen.

- Existen comentarios buenos.
- Si tu código está limpio entonces no debería tener ningún comentario.
- Es una buena práctica comentar todos los métodos de nuestras clases con una descripción de lo que hacen.
- Todo comentario es bueno pues nos entrega información extra sobre el código.

- Se recomienda tener archivos cortos de código.

- Se recomienda tener archivos cortos de código.
- Se recomienda dejar los métodos de bajo nivel (con detalles de implementación) al final del archivo.

- Se recomienda tener archivos cortos de código.
- Se recomienda dejar los métodos de bajo nivel (con detalles de implementación) al final del archivo.
- Se recomienda evitar usar atributos protegidos porque hacen que conceptos relacionados se encuentren en archivos distintos.

Patrones de Diseño

**¿Qué saben sobre
patrones de diseño?**

Patrones de Diseño

Patrones de diseño:

Son soluciones típicas para problemas recurrentes en diseño de software. Son diseños prehechos que puedes ajustar para resolver problemas recurrentes en tu código.

Patrones de Diseño

Patrones de diseño:

Son soluciones típicas para problemas recurrentes en diseño de software. Son diseños prehechos que puedes ajustar para resolver problemas recurrentes en tu código.

Importante:

A diferencia de *clean code* no siempre es buena idea usar un patrón de diseño. Forzar un patrón hace que termines con códigos complejos que generalizan en aspectos que no agregan valor a tu software. La complejidad mata el alma de los ingenieros de software.

Patrones de Diseño

Patrones de diseño:

Son soluciones típicas para problemas recurrentes en diseño de software. Son diseños prehechos que puedes ajustar para resolver problemas recurrentes en tu código.



Patrones de diseño:

Son soluciones típicas para problemas recurrentes en diseño de software. Son diseños prehechos que puedes ajustar para resolver problemas recurrentes en tu código.

Importante:

A diferencia de *clean code* no siempre es buena idea usar un patrón de diseño. Forzar un patrón hace que termines con códigos complejos que generalizan en aspectos que no agregan valor a tu software. La complejidad mata el alma de los ingenieros de software.

Clasificación: Existen tres clases de patrones de diseño.

- Patrones creacionales.
- Patrones estructurales.
- Patrones de comportamiento.

Recordatorio SOLID

Single-responsibility principle:

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

SOLID

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Interface segregation principle:

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Interface segregation principle:

“Clients should not be forced to depend upon interfaces that they do not use.”

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Interface segregation principle:

“Clients should not be forced to depend upon interfaces that they do not use.”

Dependency inversion principle:

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Interface segregation principle:

“Clients should not be forced to depend upon interfaces that they do not use.”

Dependency inversion principle:

“Depend upon abstractions, [not] concretions.”

Single-responsibility principle:

“There should never be more than one reason for a class to change.”

Open-closed principle:

“Software entities ... should be open for extension, but closed for modification.”

Liskov substitution principle:

“Subclasses should be substitutable for their base classes.”

Interface segregation principle:

“Clients should not be forced to depend upon interfaces that they do not use.”

Dependency inversion principle:

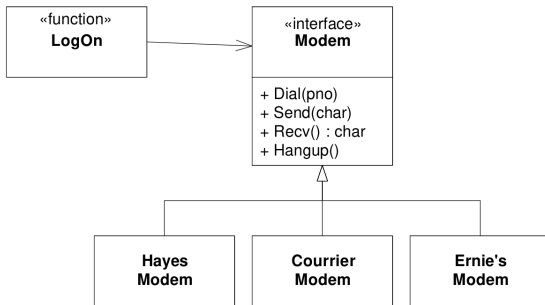
“Depend upon abstractions, [not] concretions.”

The Open Closed Principle (OCP)

OCP: *“A module should be open for extension but closed for modification.”*

The Open Closed Principle (OCP)

OCP: *"A module should be open for extension but closed for modification."*



La clave para lograr OCP es usando herencia y polimorfismo.

Observación:

**Observación:
OCP es una mentira**

The Open Closed Principle (OCP)

OCP: *“A module should be open for extension but closed for modification.”*

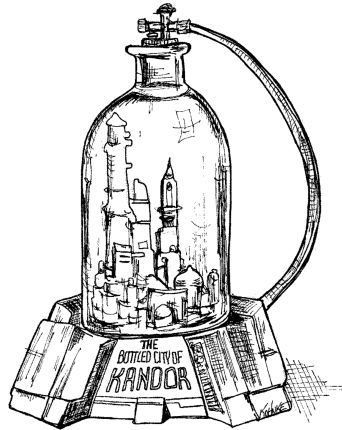
The Open Closed Principle (OCP)

OCP: *“A module should be open for extension but closed for modification.”*

El problema es que en algún lado hay que crear objetos de la nueva clase para poder usarla. Ahí es donde OCP se rompe. Con OCP igual hay que modificar las clases donde se **crean objetos**.

Systems

by Dr. Kevin Dean Wampler



*"Complexity kills. It sucks the life out of developers,
it makes products difficult to plan, build, and test."*

—Ray Ozzie, CTO, Microsoft Corporation

The Open Closed Principle (OCP)

Mensaje: Debemos encapsular las clases que crean objetos y separar su lógica de las clases que usan los objetos. Esto permite que OCP funcione en la práctica.

The Open Closed Principle (OCP)

Mensaje: Debemos encapsular las clases que crean objetos y separar su lógica de las clases que usan los objetos. Esto permite que OCP funcione en la práctica.

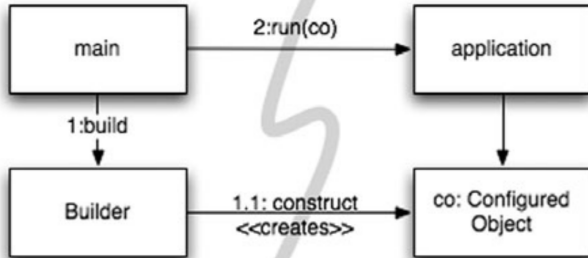


Figure 11-1

Separating construction in `main()`

Ejercicio (tipo examen)

Simulando la vida de un honguito

Se tiene un tablero con honguitos que nacen, sobreviven, se reproducen y mueren. Existen distintos tipos de honguitos y queremos crear un simulador que funcione para cualquier honguito. Esto es común para todo honguito:

- Los honguitos mueren dependiendo de diversas circunstancias.
- Mientras vivan, los honguitos lanzan esporas (que duran un turno).
- Dependiendo de la cantidad de esporas que caigan en una casilla vacía nacerá un honguito en esa posición.

Simulando la vida de un honguito

Se tiene un tablero con honguitos que nacen, sobreviven, se reproducen y mueren. Existen distintos tipos de honguitos y queremos crear un simulador que funcione para cualquier honguito. Esto es común para todo honguito:

- Los honguitos mueren dependiendo de diversas circunstancias.
- Mientras vivan, los honguitos lanzan esporas (que duran un turno).
- Dependiendo de la cantidad de esporas que caigan en una casilla vacía nacerá un honguito en esa posición.

Honguito clásico:

- Sobrevive solo si tiene 2 o 3 vecinos.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Simulando la vida de un honguito

Se tiene un tablero con honguitos que nacen, sobreviven, se reproducen y mueren. Existen distintos tipos de honguitos y queremos crear un simulador que funcione para cualquier honguito. Esto es común para todo honguito:

- Los honguitos mueren dependiendo de diversas circunstancias.
- Mientras vivan, los honguitos lanzan esporas (que duran un turno).
- Dependiendo de la cantidad de esporas que caigan en una casilla vacía nacerá un honguito en esa posición.

	O	O	
O	O	O	
	O	O	

Simulando la vida de un honguito

Se tiene un tablero con honguitos que nacen, sobreviven, se reproducen y mueren. Existen distintos tipos de honguitos y queremos crear un simulador que funcione para cualquier honguito. Esto es común para todo honguito:

- Los honguitos mueren dependiendo de diversas circunstancias.
- Mientras vivan, los honguitos lanzan esporas (que duran un turno).
- Dependiendo de la cantidad de esporas que caigan en una casilla vacía nacerá un honguito en esa posición.

O		O	
O			O
O		O	

Simulando la vida de un honguito

Se tiene un tablero con honguitos que nacen, sobreviven, se reproducen y mueren. Existen distintos tipos de honguitos y queremos crear un simulador que funcione para cualquier honguito. Esto es común para todo honguito:

- Los honguitos mueren dependiendo de diversas circunstancias.
- Mientras vivan, los honguitos lanzan esporas (que duran un turno).
- Dependiendo de la cantidad de esporas que caigan en una casilla vacía nacerá un honguito en esa posición.

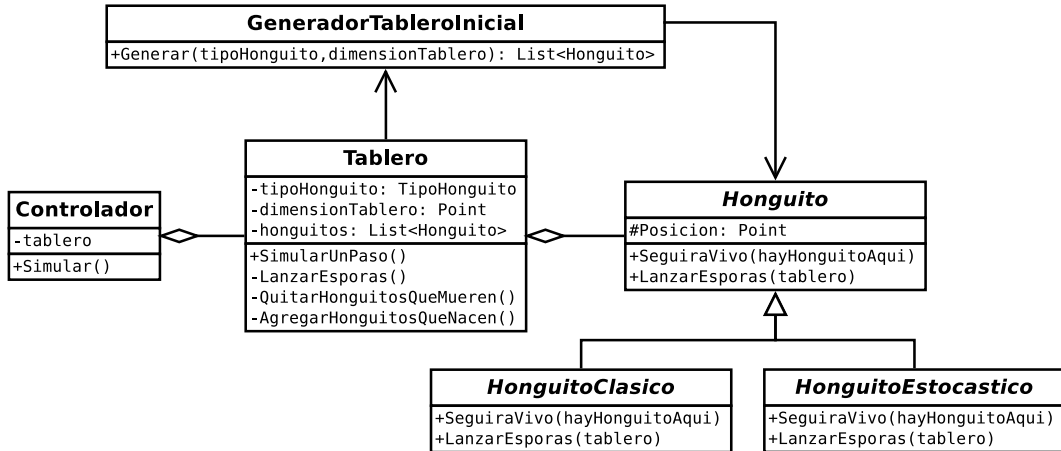
Honguito estocástico:

- En cada turno, sobrevive con probabilidad 0.2.
- Cada turno lanzan 15 esporas que se reparten siguiendo una distribución normal.
- La probabilidad de que nazca un honguito depende de la cantidad de esporas que caen en una casilla vacía.

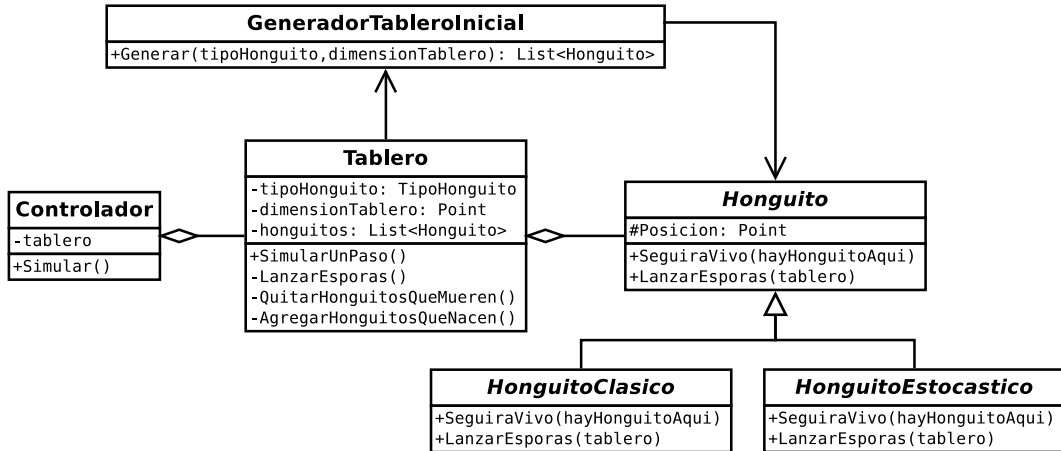
Simulando la vida de un honguito

Rodrigo Toro programó un simulador, pero no me gustó :/

Simulando la vida de un honguito



Simulando la vida de un honguito



Lo que no me gusta es que agregar un nuevo honguito obliga a cambiar las clases que crean nuevos honguitos (Tablero y GeneradorTableroInicial)

Simulando la vida de un honguito

¿Cómo funciona la creación de honguitos?

```
2 public enum TipoHonguito { Clasico, Estocastico }
```

Simulando la vida de un honguito

¿Cómo funciona la creación de honguitos?

```
2 public enum TipoHonguito { Clasico, Estocastico }
```

En el main indico el tipo de honguito que quiero que se creen en la simulación.

```
2 int dimX = 25;
3 int dimY = 10;
4 TipoHonguito tipoHonguito = TipoHonguito.Clasico;
5 Point dimTab = new Point(dimX, dimY);
6
7 Controlador controlador = new Controlador(tipoHonguito, dimTab);
8 controlador.Simular();
```

El problema es que después hay que crear honguitos usando ifs sobre tipoHonguito.

Simulando la vida de un honguito

De hecho, la clase GeneradorTableroInicial tiene este método:

```
2 private static void AgregarHonguitoSegunTipo(Point posicion) {  
3     Honguito honguito;  
4     if (tipoHonguito == TipoHonguito.Clasico)  
5         honguito = new HonguitoClasico(posicion, dimensionTablero);  
6     else if (tipoHonguito == TipoHonguito.Estocastico)  
7         honguito = new HonguitoEstocastico(posicion, dimensionTablero);  
8     else  
9         throw new NotImplementedException("El honguito no existe.");  
10    honguitos.Add(honguito);  
11 }
```


Simulando la vida de un honguito

La situación en Tablero es aún peor:

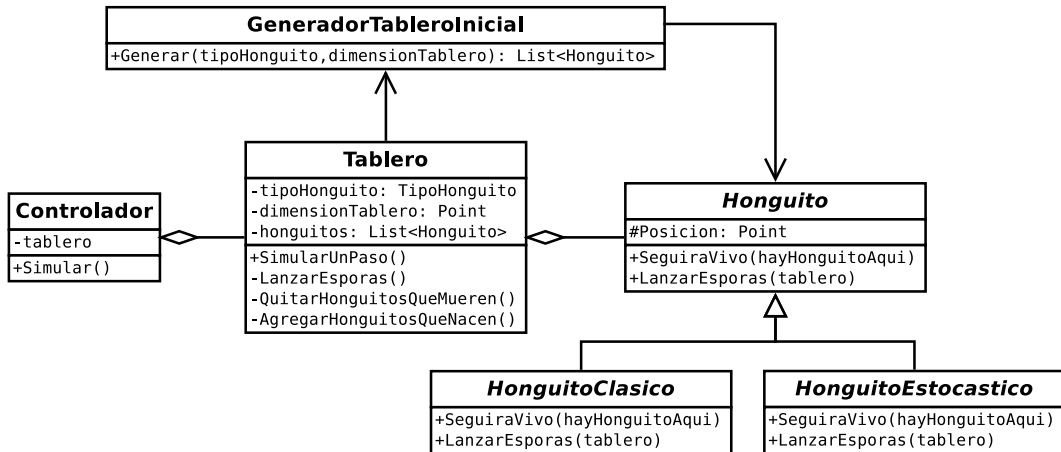
```
2 private void AgregarHonguitoSiCorresponde(int x, int y){  
3     if (tipoHonguito == TipoHonguito.Clasico)  
4         AgregarHonguitoClasicoSiCorresponder(x, y);  
5     else if (tipoHonguito == TipoHonguito.Estocastico)  
6         AgregarHonguitoEstocasticoSiCorresponder(x, y);  
7     else  
8         throw new NotImplementedException("El honguito no existe.");  
9 }
```

Simulando la vida de un honguito

La situación en Tablero es aún peor:

```
2 private void AgregarHonguitoClasicoSiCorresponder(int x, int y) {
3     bool debeNacer = conteoEsporas[x, y] == 3;
4     if (!hayHonguitoAqui[x, y] && debeNacer)
5         honguitos.Add(new HonguitoClasico(new Point(x, y), dimTab));
6 }
7
8 private void AgregarHonguitoEstocasticoSiCorresponder(int x, int y){
9     double probabilidadDeNacer = conteoEsporas[x, y] / 10.0;
10    bool debeNacer = MuestrearEvento(probabilidadDeNacer);
11    if (!hayHonguitoAqui[x, y] && debeNacer)
12        honguitos.Add(new HonguitoEstocastico(new Point(x, y), dimTab));
13 }
```

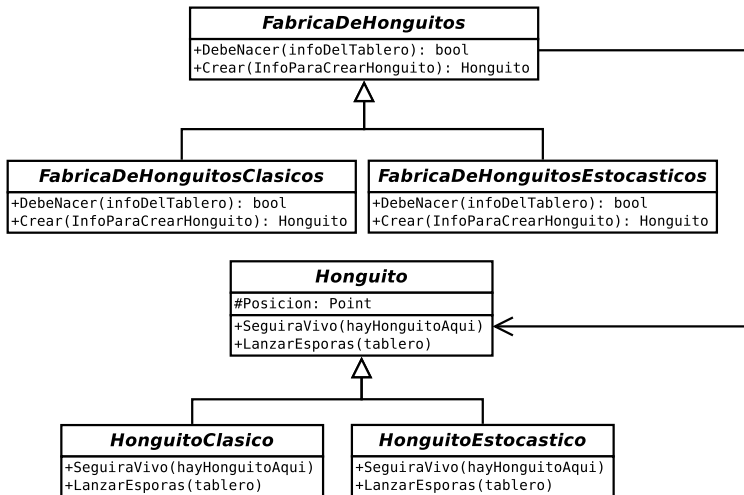
Simulando la vida de un honguito



¿Cómo podemos extender Honguito sin modificar ninguna clase (salvo por el main)?

Solución

Crear una fábrica de honguitos.



La fábrica abstracta define la interfaz de la fábrica:

```
2 public abstract class FabricaDeHonguitos{  
3     public abstract bool DebeNacer(Point pos, bool[,] honguitos, int  
4     [,] esporas);  
5     public abstract Honguito Crear(Point pos, Point dimTab);  
}
```

La fábrica abstracta define la interfaz de la fábrica:

```
2 public abstract class FabricaDeHonguitos{
3     public abstract bool DebeNacer(Point pos, bool[,] honguitos, int
4     [,] esporas);
5     public abstract Honguito Crear(Point pos, Point dimTab);
6 }
```

Luego cada fábrica construye objetos del tipo que le corresponda:

```
2 public class FabricaDeHonguitosClasicos : FabricaDeHonguitos {
3     public override bool DebeNacer(Point pos, bool[,] honguitos, int
4     [,] esporas){
5         bool hayTresEsporas = esporas[pos.X, pos.Y] == 3;
6         bool laCasillaEstaVacía = !honguitos[pos.X, pos.Y];
7         return laCasillaEstaVacía && hayTresEsporas;
8     }
9     public override Honguito Crear(Point pos, Point dimTab)
10    => new HonguitoClasico(pos, dimTab);
11 }
```

La gracia es que después la aplicación crea honguitos usando la fábrica que le demos:

```
2 int dimX = 25;
3 int dimY = 10;
4 FabricaDeHonguitos fabrica = new FabricaDeHonguitosEstocasticos();
5 Point dimTab = new Point(dimX, dimY);
6
7 Controlador controlador = new Controlador(fabrica, dimTab);
8 controlador.Simular();
```


La gracia es que después la aplicación crea honguitos usando la fábrica que le demos:

```
2 int dimX = 25;
3 int dimY = 10;
4 FabricaDeHonguitos fabrica = new FabricaDeHonguitosEstocasticos();
5 Point dimTab = new Point(dimX, dimY);
6
7 Controlador controlador = new Controlador(fabrica, dimTab);
8 controlador.Simular();
```

... y el código ya no necesita esos ifs que dependen de cada tipo.

El código del GeneradorTableroInicial:

```
2 private static void AgregarHonguitoSegunTipo(Point posicion) {  
3     Honguito honguito;  
4     if (tipoHonguito == TipoHonguito.Clasico)  
5         honguito = new HonguitoClasico(posicion, dimensionTablero);  
6     else if (tipoHonguito == TipoHonguito.Estocastico)  
7         honguito = new HonguitoEstocastico(posicion, dimensionTablero);  
8     else  
9         throw new NotImplementedException("El honguito no existe.");  
10    honguitos.Add(honguito);  
11 }
```

El código del GeneradorTableroInicial:

```
2 private static void AgregarHonguitoSegunTipo(Point pos) {  
3     Honguito honguito = fabricaDeHonguitos.Crear(pos, dimTab);  
4     honguitos.Add(honguito);  
5 }
```

El código del GeneradorTableroInicial:

```
2 private static void AgregarHonguitoSegunTipo(Point pos) {  
3     Honguito honguito = fabricaDeHonguitos.Crear(pos, dimTab);  
4     honguitos.Add(honguito);  
5 }
```

El código del Tablero:

```
2 private void AgregarHonguitoSiCorresponde(int x, int y){  
3     if (tipoHonguito == TipoHonguito.Clasico)  
4         AgregarHonguitoClasicoSiCorresponder(x, y);  
5     else if (tipoHonguito == TipoHonguito.Estocastico)  
6         AgregarHonguitoEstocasticoSiCorresponder(x, y);  
7     else  
8         throw new NotImplementedException("El honguito no existe.");  
9 }
```

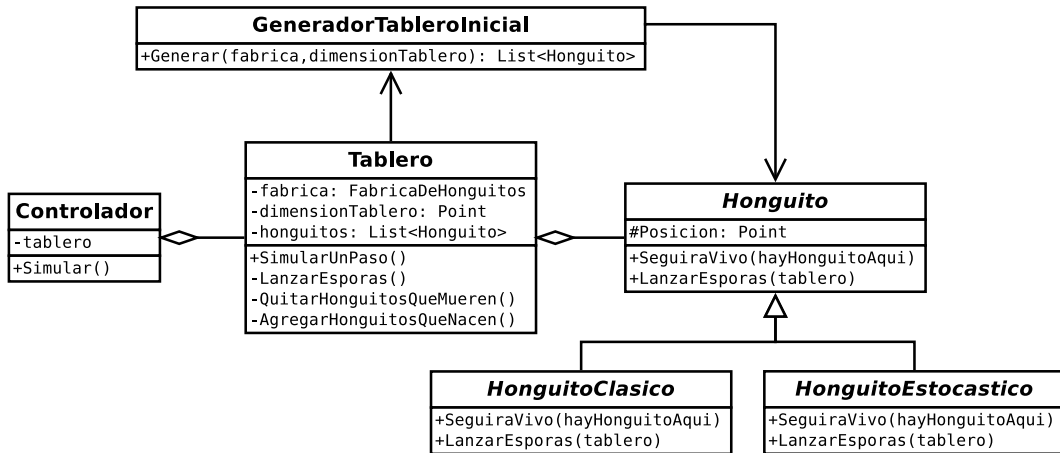
El código del GeneradorTableroInicial:

```
2 private static void AgregarHonguitoSegunTipo(Point pos) {  
3     Honguito honguito = fabricaDeHonguitos.Crear(pos, dimTab);  
4     honguitos.Add(honguito);  
5 }
```

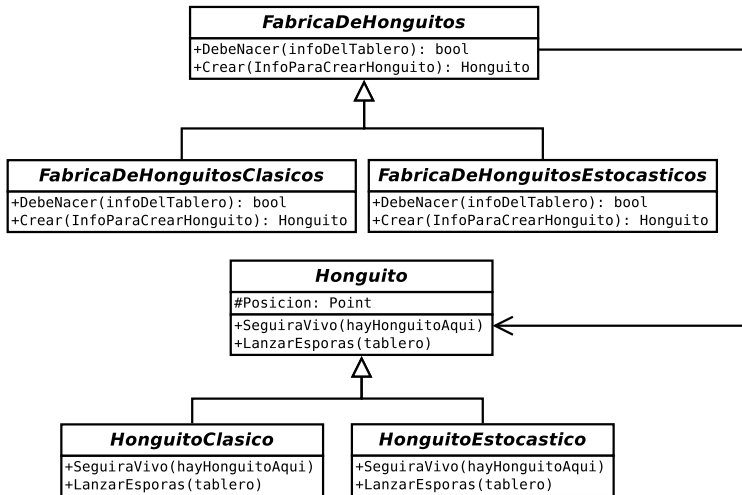
El código del Tablero:

```
2 private void AgregarHonguitoSiCorresponde(int x, int y){  
3     Point pos = new Point(x, y);  
4     if (fabricaDeHonguitos.DebeNacer(pos, hayHonguito, esporas))  
5         honguitos.Add(fabricaDeHonguitos.Crear(pos, dimTab));  
6 }
```

¿Qué pasa si ahora quiero crear un nuevo tipo de honguito?



¿Qué pasa si ahora quiero crear un nuevo tipo de honguito?



¿Qué pasa si ahora quiero crear un nuevo tipo de honguito?

```
2 int dimX = 25;
3 int dimY = 10;
4 FabricaDeHonguitos fabrica = new FabricaDeHonguitosEstocasticos();
5 Point dimTab = new Point(dimX, dimY);
6
7 Controlador controlador = new Controlador(fabrica, dimTab);
8 controlador.Simular();
```


¿Qué pasa si ahora quiero crear un nuevo tipo de honguito?

```
2 int dimX = 25;
3 int dimY = 10;
4 FabricaDeHonguitos fabrica = new FabricaDeHonguitosEstocasticos();
5 Point dimTab = new Point(dimX, dimY);
6
7 Controlador controlador = new Controlador(fabrica, dimTab);
8 controlador.Simular();
```

Para agregar un nuevo honguito

- Extender la clase Honguito.
- Crear una nueva fábrica para el nuevo tipo de honguito.
- En el main, dar la fábrica apropiada al controlado.

Resumen

The Open Closed Principle (OCP)

Mensaje: Debemos encapsular las clases que crean objetos y separar su lógica de las clases que usan los objetos. Esto permite que OCP funcione en la práctica.

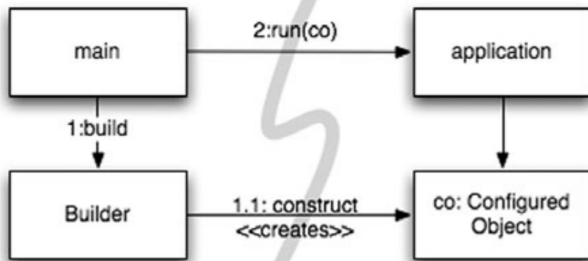


Figure 11-1

Separating construction in `main()`

The Open Closed Principle (OCP)

Mensaje: Debemos encapsular las clases que crean objetos y separar su lógica de las clases que usan los objetos. Esto permite que OCP funcione en la práctica.

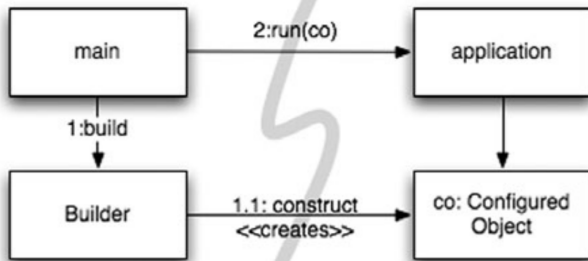


Figure 11-1

Separating construction in `main()`

Los patrones creacionales nos ayudan a separar/encapsular la lógica para crear objetos.